

Harness Control Plane

Detailed architecture and implementation specification

United States deployment baseline

AUTHORITY	Deterministic compilation, explicit permissions and signed immutable HarnessPlans
ASSURANCE	Dependency-bound evaluations, approvals, provenance and certification evidence
RELEASE	Canary promotion, pointer generations, rollback, blast-radius analysis and revocation
AZURE	Container Apps, PostgreSQL, Blob, ACR, App Configuration, Key Vault and Foundry

Planning baseline | Version 1.0 | 17 September 2026

Defines the control plane that turns foundation and KYC source packages into executable authority. Configuration values are proposed and require target-subscription validation.

Contents

Purpose and outcome	3
1. Mission, boundary, and invariants	3
2. Logical architecture	3
3. Trust zones and data classification	4
4. Authoring model and repository structure	5
5. HarnessSpec and HarnessPlan	6
6. Deterministic compiler architecture	6
7. Overlay resolution and explainability	7
8. Registry object model	8
9. PostgreSQL schema and state machines	8
10. Dependency graph and blast-radius analysis	10
11. Evaluation and certification	10
12. Signing, provenance, and trust roots	11
13. Approval and separation of duties	11
14. Release, canary, promotion, and rollback	12
15. Revocation and emergency control	12
16. APIs and events	13
17. Concurrency, idempotency, and failure recovery	13
18. Azure topology and configuration	14
19. Network and identity flows	15
20. CI/CD and infrastructure pipeline	15
21. Observability, audit, and service objectives	16
22. Resilience and disaster recovery	16
23. KYC-specific control-plane rules	17
24. Implementation plan	18
25. Prioritized engineering backlog	19
26. Verification strategy	19
27. Operational runbooks	20
28. Architecture decisions and definition of done	20

Purpose and outcome

The Harness Control Plane converts reviewed human-authored packages into immutable, evaluated, signed, and releasable runtime authority. It is the system that proves what a KYC workflow may do before the Execution Control Plane runs it. The United States baseline in this document assumes separate non-production and production Azure subscriptions, US data residency, Microsoft Entra workload identities, private networking, and the governance controls described in the foundation and KYC implementation plan.

This design deliberately separates three registries. Git stores authoring history and pull-request evidence. The Harness Registry stores executable authority, immutable artifacts, dependency relationships, attestations, releases, and revocations. An enterprise agent registry such as Microsoft 365 Agent Registry records inventory, ownership, and organizational governance. Only a released and non-revoked HarnessPlan digest authorizes runtime behavior.

1. Mission, boundary, and invariants

The Harness Control Plane owns the path from source package to production release. Its output is a signed release envelope that binds a canonical HarnessPlan digest to every dependency needed to execute it: policies, agent and tool definitions, schemas, model profiles, evaluation attestations, container artifacts, migration definitions, and regional constraints.

It does not own live case state, task dispatch, worker leases, evidence collection, or the commit of business effects. Those belong to the Execution Control Plane. It does not provide a document-of-record for customers or cases. It also does not treat a model deployment, container tag, Git branch, or mutable configuration key as runtime authority.

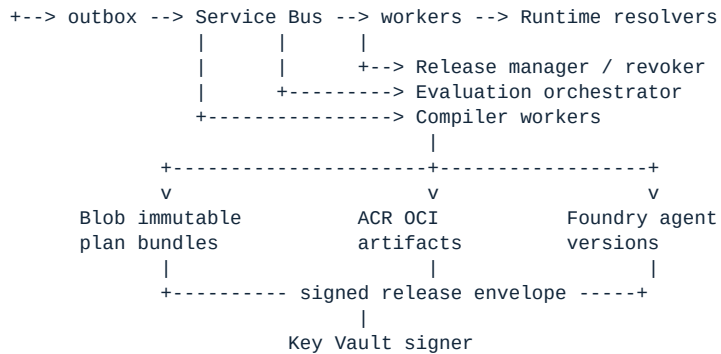
The following invariants are enforced in code and Azure policy:

- A runtime starts work only from a released HarnessPlan digest with a valid signature and no effective revocation.
- Compilation is deterministic: the same source tree, compiler digest, dependency lock, and build parameters produce the same canonical plan digest.
- All executable dependencies are immutable and referenced by digest or immutable version. The string `latest` is rejected.
- Promotion changes a small active-release pointer; it never rewrites an artifact.
- Each case pins its plan, policy, schema, model profile, tool, and skill versions. Pointer changes apply to new cases unless an approved migration says otherwise.
- Policy evaluation, data-flow checks, permission derivation, evaluation, approval, signing, and release remain distinct stages with separate evidence.
- A compiler cannot approve its own output, and a release manager cannot create a signature without an approved evaluation attestation.
- Revocation can immediately fence new dispatches and result commits even when a case is pinned to the revoked digest.
- Every allow or deny result is explainable from source location, overlay lineage, dependency version, rule identifier, and evidence.

2. Logical architecture

The control plane is an asynchronous set of services around authoritative PostgreSQL records and immutable artifacts. Synchronous APIs accept commands and return operation identifiers. Workers use an outbox and Azure Service Bus to advance long-running builds, evaluations, deployments, and releases.





Component	Responsibility	Scale and isolation
Control API	Package, build, evaluation, approval, release, pointer, and revocation commands; registry queries	Azure Container Apps; min 2 production replicas; internal ingress behind APIM
Compiler worker	Parse, validate, resolve, qualify, compile, canonicalize, and produce unsigned artifacts	Dedicated Container Apps job profile; no production data access; pinned compiler image
Evaluation orchestrator	Construct evaluation run, dispatch suites, collect metrics and evidence, issue candidate attestation	Separate workload identity; calls test environments only
Agent deployer	Build and push ACR artifacts; create immutable Foundry agent versions; capture deployed identifiers	Dedicated identity scoped to approved registries and Foundry project
Release manager	Validate approvals and attestations, request signing, create release, promote pointer, roll back	Separate identity and deployment; production commands require step-up role
Revocation service	Publish high-priority revocations and notify runtime resolvers	Minimum two replicas; independent queue and alert path
Registry projector	Maintain dependency graph, searchable projections, App Configuration snapshots, and regional replicas	Idempotent consumers; rebuildable from the event log
PostgreSQL	Authoritative metadata, state machines, locks, outbox, approvals, and audit sequence	Zone-redundant production server with private endpoint
Blob Storage	Immutable source bundles, canonical plans, reports, evidence manifests, SBOMs, and provenance	Versioning, soft delete, immutability policies for released artifacts
Azure Container Registry	OCI images and OCI artifact bundles addressed by digest	Premium, private endpoint, zone redundancy where supported, geo-replication if required
Key Vault or Managed HSM	Signing keys and signature operations	Private endpoint; RBAC; purge protection; signer identity only

3. Trust zones and data classification

There are four trust zones. The authoring zone contains developer workstations, Git hosting, pull requests, and CI. Source is untrusted until validation. The build zone compiles and evaluates packages without access to production customer data. The release zone contains signing keys, approvals, release records, and production pointer mutation. The runtime zone consumes signed plans and exposes only minimal status back to the registry.

The build zone uses synthetic and approved de-identified KYC evaluation data. Production PII, identity documents, biometric images, Social Security numbers, and live vendor responses are prohibited. Evaluation evidence contains scenario identifiers, hashes, metrics, redacted traces, and result classifications. Raw prompts or outputs are retained only when the suite explicitly permits them and the storage classification supports them.

Data	Classification	Allowed stores	Retention baseline
Source package and schemas	Internal	Git, Blob	Repository policy plus released-artifact retention

Data	Classification	Allowed stores	Retention baseline
Canonical HarnessPlan	Internal controlled	Blob, registry cache	Release life plus seven years
Signature, provenance, SBOM	Audit	PostgreSQL, Blob	Seven years minimum
Synthetic evaluation inputs	Internal test	Blob evaluation account	One year or suite policy
Redacted evaluation traces	Confidential	Blob, Log Analytics with field controls	180 days hot; archive per compliance
Approval and release audit	Audit	PostgreSQL and immutable export	Seven years minimum
Production customer data	Restricted	Prohibited in control-plane build and evaluation services	Not applicable

4. Authoring model and repository structure

A domain package is the smallest independently versioned source unit. The KYC package owns workflow intent, jurisdiction overlays, policy mappings, schemas, evaluations, and migration definitions. Foundation packages provide shared capabilities, schemas, policies, and profiles. A package manifest declares every dependency and compatibility range; the generated lock file resolves each range to an immutable digest.

```
/packages
  /foundation
    /capabilities
    /schemas
    /policies
    /model-profiles
    /execution-profiles
  /domains/kyc-us
    harness.yaml
    package.yaml
    /agents
    /skills
    /tools
    /policies
    /schemas
    /prompts
    /workflows
    /evaluations
      /golden
      /adversarial
      /metamorphic
      /fault
    /migrations
    /overlays
      /jurisdiction/us
      /environment/nonprod
      /environment/prod
  /locks
  /compiler
  /registry-api
  /release
  /infra
```

`package.yaml` includes package name, semantic version, owners, risk class, minimum compiler version, dependencies, allowed regions, data classes, evaluation requirements, and compatibility claims. `harness.yaml` is the human-readable entry point. It can reference declared package objects but cannot embed executable code, scripts, network addresses, or secrets.

Every pull request runs formatting, safe YAML parsing, schema validation, dependency resolution, static authority analysis, data-flow analysis, workflow checks, policy tests, schema compatibility checks, and the fast evaluation suite. Protected branches require CODEOWNERS review from the KYC domain and platform control owners when a change expands authority.

5. HarnessSpec and HarnessPlan

HarnessSpec expresses intent. HarnessPlan is the normalized runtime contract. A plan has no templates, inheritance, environment substitution, unbounded version ranges, or unresolved names. Every reference is a stable object identifier plus digest. Every action includes its derived permissions, required evidence, allowed data classes, budget, timeout, retry class, and assurance conditions.

```
apiVersion: harness.factory/v1
kind: HarnessSpec
metadata:
  name: kyc-us-individual
  version: 1.4.0
spec:
  workflow: kyc.individual.v3
  overlays: [jurisdiction.us, environment.prod]
  capabilities:
    - identity.document.verify
    - identity.ssn.verify
    - screening.sanctions.search
  policies:
    - kyc.us.acceptance.v7
  assurance:
    minimum: high
  outcomes: [approved, rejected, manual_review]
```

The corresponding plan contains resolved workflow nodes, input and output schema digests, concrete capability providers, policy bundle digest, permissions, evidence producers, side-effect classes, release constraints, and a dependency closure. The compiler serializes the plan using a documented canonical JSON profile, hashes the exact bytes with SHA-256, and stores both bytes and digest. A digest is never calculated over display YAML.

Required plan envelope fields are planId, planDigest, packageDigest, compilerDigest, dependencyLockDigest, policyDigest, schemaSetDigest, modelProfileDigests, toolDigests, skillDigests, evaluationRequirementSet, regionSet, dataClassSet, createdAt, and buildProvenanceDigest. The signed release envelope later adds approval, attestation, signature, release, and activation data.

6. Deterministic compiler architecture

The compiler runs as a pinned OCI image with a read-only root filesystem, no general internet access, a fixed locale and timezone, and explicit resource limits. It accepts a source-bundle digest, dependency-lock digest, compiler digest, target overlay set, and build nonce used only for operation correlation. Time, nonce, worker hostname, and database identifiers cannot enter canonical output.

Stage	Input and output	Enforced rules
1. Safe parse	Source bytes to typed syntax tree	Duplicate keys, aliases, custom tags, executable YAML, invalid UTF-8, excessive nesting, and oversized fields fail
2. Schema validate	Typed objects	API version, required fields, enum values, ownership, risk class, and naming rules
3. Resolve dependencies	Manifest and lock to immutable closure	No latest, branches, mutable tags, unresolved ranges, hidden transitive dependencies, or digest mismatch
4. Apply overlays	Base objects to resolved objects plus lineage	Fixed precedence; broadening authority requires explicit entitlement and approval class
5. Qualify capabilities	Abstract capability to provider/version	Region, assurance, data class, availability, certification, and interface compatibility
6. Compile policy	Policy source to executable decision bundle	Deterministic rule set, rule IDs, obligations, denial reasons, and test coverage

Stage	Input and output	Enforced rules
7. Derive permissions	Workflow plus providers to least privilege	Each node gets explicit commands, resources, scopes, data classes, and effect classes
8. Analyze data flow	Graph edges and schemas	Residency, purpose, retention, model-input, telemetry, and vendor-transfer restrictions
9. Analyze workflow	Directed graph	Cycles, missing producers, unreachable outcomes, orphan nodes, unbounded fan-out, and invalid compensation fail
10. Check evidence	Outcome and policy obligations	Every claim and decision has an acceptable evidence producer and schema
11. Check assurance	Agent, model, tool, skill combination	Requested assurance cannot exceed weakest certified dependency
12. Check migration	Existing released plans and proposed definitions	State mapping, evidence reuse, policy compatibility, rollback, and cohort constraints
13. Bound execution	Plan graph	Time, cost, token, call, retry, concurrency, and side-effect budgets
14. Canonicalize	Fully resolved object	Stable key order, numeric and string encoding, set ordering, and null rules
15. Digest and package	Canonical bytes and reports	Create plan digest, manifest, authority report, graph, and compile attestation candidate

The compiler emits diagnostics as stable machine-readable records: code, severity, object path, source file and line, overlay lineage, dependency identity, rule identifier, and remediation hint. A future compiler version may add diagnostics but cannot silently reinterpret an existing API version. Semantic changes require a new compiler compatibility version and replay of affected packages.

```

compile(request):
  source = verify_blob_digest(request.source_digest)
  lock = verify_dependency_lock(request.lock_digest)
  ast = safe_parse(source)
  objects = schema_validate(ast)
  closure = resolve_immutable(objects, lock)
  resolved, lineage = apply_overlays(closure, request.overlay_set)
  qualified = qualify_capabilities(resolved)
  policy = compile_policy(qualified)
  permissions = derive_least_privilege(qualified, policy)
  assert_data_flows(qualified, permissions)
  assert_workflow_graph(qualified)
  assert_evidence_and_assurance(qualified)
  assert_migration_compatibility(qualified)
  bounded = attach_budgets(qualified, permissions)
  bytes = canonical_json(bounded)
  return sha256(bytes), bytes, diagnostics, dependency_graph

```

7. Overlay resolution and explainability

Configuration resolves in this fixed order: platform, domain, jurisdiction, business unit, tenant, environment, and case policy context. Later overlays may tighten timeouts, budgets, data access, eligibility, or required evidence. An overlay that broadens a capability, data class, side effect, provider, outcome, or budget must declare `authorityExpansion`, cite an entitlement, and receive the approval class associated with the expanded risk.

The compiler writes an origin map for every resolved field. An explain request for `/workflow/nodes/document_check/timeoutSeconds` returns the base value, each overlay operation, the final value, the authority classification, and source locations. The release UI uses this data to show semantic differences between two plan digests rather than a raw text diff.

Overlay operations are restricted to `set`, `remove`, `appendUnique`, `intersect`, and `tighten`. Arbitrary patch scripts and environment-variable interpolation are prohibited. Collection ordering rules are part of the schema. A conflict without an

explicit resolution rule fails compilation.

8. Registry object model

The Harness Registry stores immutable objects and mutable lifecycle records. Immutable objects include source bundles, compiled plans, policy bundles, schemas, tools, skills, prompts, model profiles, execution profiles, evaluation suites, capability definitions, migration definitions, SBOMs, and provenance statements. Lifecycle records include builds, evaluation runs, attestations, approvals, releases, active pointers, deprecations, and revocations.

Object	Stable identity	Required metadata
PackageVersion	package name + semantic version + digest	Owner, risk class, source commit, dependency lock, compiler minimum
HarnessPlan	plan digest	Package digest, compiler digest, overlay set, dependency closure, authority report
CapabilityVersion	capability name + version + digest	Interface schemas, providers, regions, assurance, data classes, certification
ToolVersion	tool name + version + digest	Commands, effects, schemas, scopes, network destinations, image digest
SkillVersion	skill name + version + digest	Inputs, outputs, required tools, evidence, budgets, evaluation suite
ModelProfileVersion	profile name + version + digest	Deployment class, region, data policy, allowed tasks, fallback rules
EvaluationAttestation	attestation ID + content digest	Subject digests, suite digests, metrics, thresholds, environment, expiry
Release	release ID	Plan digest, approvals, attestation, signature, state, cohort, region
ActivePointer	scope + generation	Release ID, plan digest, activation time, ETag, prior generation
Revocation	revocation ID	Subject digest, scope, effective time, reason, replacement, issuer

Artifact metadata never substitutes for content verification. A reader obtains the expected digest from the signed envelope, downloads the bytes, computes the digest locally, verifies the signature and trust chain, and checks revocation status before caching.

9. PostgreSQL schema and state machines

Use one Azure Database for PostgreSQL Flexible Server per environment boundary, with separate databases or schemas for registry metadata and operational jobs. Production uses zone-redundant high availability, private access, Microsoft Entra authentication, customer-managed backup controls where required, and geo-redundant backup to the approved paired US region.

```
create table artifact (
  artifact_digest text primary key,
  artifact_type text not null,
  media_type text not null,
  uri text not null,
  size_bytes bigint not null,
  created_at timestamptz not null,
  provenance_digest text not null,
  metadata jsonb not null
);

create table build (
  build_id uuid primary key,
  package_digest text not null,
```

```

compiler_digest text not null,
lock_digest text not null,
overlay_set_digest text not null,
state text not null,
plan_digest text,
diagnostics_uri text,
idempotency_key text not null unique,
row_version bigint not null default 0,
created_at timestamptz not null,
updated_at timestamptz not null
);

create table dependency_edge (
  subject_digest text not null,
  predicate text not null,
  object_digest text not null,
  edge_metadata jsonb not null,
  primary key (subject_digest, predicate, object_digest)
);

create table evaluation_attestation (
  attestation_id uuid primary key,
  subject_digest text not null,
  suite_set_digest text not null,
  result_digest text not null,
  status text not null,
  issued_at timestamptz not null,
  expires_at timestamptz,
  environment_digest text not null,
  signature_uri text
);

create table release_record (
  release_id uuid primary key,
  plan_digest text not null,
  attestation_id uuid not null,
  state text not null,
  signature_uri text,
  cohort jsonb not null,
  activation_time timestamptz,
  previous_release_id uuid,
  idempotency_key text not null unique,
  row_version bigint not null default 0
);

create table active_pointer (
  scope_key text primary key,
  generation bigint not null,
  release_id uuid not null,
  plan_digest text not null,
  activation_time timestamptz not null,
  etag uuid not null,
  prior_release_id uuid
);

create table revocation (
  revocation_id uuid primary key,
  subject_digest text not null,
  scope jsonb not null,
  effective_at timestamptz not null,
  reason_code text not null,
  replacement_digest text,
  issued_by text not null,
  signature_uri text not null,
  unique (subject_digest, effective_at)
);

```

Build states are QUEUED, RESOLVING, COMPILING, SUCCEEDED, FAILED, and CANCELLED. Evaluation states are QUEUED, PROVISIONING, RUNNING, ASSESSING, PASSED, FAILED, EXPIRED, and CANCELLED. Release states are DRAFT, VALIDATED, COMPILED, EVALUATED, APPROVED, SIGNED, RELEASED, CANARY, PRODUCTION, DEPRECATED, and REVOKED. State changes use compare-and-swap on row_version, append an audit event, and write an outbox record in the same transaction.

10. Dependency graph and blast-radius analysis

Every artifact is a vertex and every typed dependency is an edge. Predicates include `COMPILED_FROM`, `DEPENDS_ON`, `USES_POLICY`, `USES_SCHEMA`, `USES_TOOL`, `USES_SKILL`, `USES_MODEL_PROFILE`, `PROVIDES_CAPABILITY`, `EVALUATED_BY`, `DEPLOYED_AS`, `SIGNED_BY`, `SUPERSEDES`, and `MIGRATES_FROM`.

The graph supports release gates and incident response. A proposed tool revocation queries all plans that transitively depend on its digest, then identifies active pointers, pinned cases, regions, tenants, and migration paths. A model-profile change identifies evaluation suites and certifications that must be rerun. A schema change identifies consumers, producers, stored evidence, and migration compatibility.

PostgreSQL adjacency tables and recursive queries are sufficient initially. Materialized transitive-closure projections are maintained for common blast-radius queries. Adopt a graph database only if measured query latency or relationship volume justifies it; the source of truth remains the append-only registry model.

11. Evaluation and certification

Evaluation determines whether a precise dependency closure is fit for a release scope. It does not certify a name or mutable endpoint. An attestation binds the plan digest, compiler digest, policy digest, tool and skill digests, model profile and deployment class, suite digests, dataset digests, evaluator image digest, environment digest, metrics, thresholds, exceptions, and expiry.

Layer	Purpose	Release gate example
Schema and contract	Validate types, compatibility, required evidence, and error contracts	Zero breaking consumer errors
Tool	Command allowlists, side effects, retries, timeouts, redaction, and failure mapping	100 percent restricted-command denial
Skill	Input/output, evidence, deterministic rules, and bounded model use	Required evidence completeness at threshold
Agent	Delegation, stop conditions, prompt-injection resistance, and escalation	No unauthorized tool call in adversarial set
Policy	Golden allow/deny/obligation decisions	100 percent high-risk policy cases pass
Workflow	Paths, outcomes, compensation, and manual review	All reachable terminal outcomes covered
Golden KYC	Known identities, documents, sanctions, PEP, fraud, and edge cases	Domain-approved accuracy thresholds
Adversarial	Prompt injection, forged evidence, schema confusion, and tool coercion	Zero critical control bypasses
Metamorphic	Equivalent input transformations preserve required outcome	No unexplained high-severity variance
Fault	Vendor timeout, throttling, stale data, partial response, and region failure	Safe retry or manual-review behavior
Replay and shadow	Compare candidate with approved history using de-identified fixtures	Difference budget within approved bounds
Canary	Restricted production cohort with guardrails	Error, latency, cost, and outcome guardrails hold

Foundry evaluations may run structured agent tests, but the Harness evaluation manifest remains the authoritative suite definition and is versioned with the package. Microsoft documents that hosted agent deployments create immutable agent versions and that agent versions move through test, evaluation, publish, and monitor stages. The control plane records the resulting Foundry version as a dependency rather than treating a mutable agent name as authority: [Agent development lifecycle](#), [Deploy a hosted agent](#), and [Test a hosted agent](#).

An attestation expires when its declared date is reached or any bound dependency is revoked. Changes classified as documentation-only may reuse an attestation only if the compiler proves that the canonical plan digest and evaluated dependency closure are unchanged.

12. Signing, provenance, and trust roots

After approvals and a passing attestation, the release manager builds a canonical release envelope. It contains the plan digest, complete dependency manifest digest, attestation digest, provenance digest, release scope, activation constraints, signer key identifier and version, and signature algorithm. The signer identity sends the envelope hash to Azure Key Vault. Microsoft states that Key Vault sign and verify operations operate on a digest and that the private key remains protected in the service: [Key Vault keys](#).

Use a Premium Key Vault HSM-backed key for the initial production design. Use Managed HSM when a control requires single-tenant HSM service and FIPS 140-3 Level 3 validation: [Managed HSM overview](#). Enable soft delete and purge protection, use private endpoints, disable public network access after bootstrap, rotate by key version, and retain old public keys for verification through artifact retention.

Compiler images, evaluator images, and tool images are stored in ACR by digest. Sign OCI artifacts with Notation and verify them before build or deployment. Microsoft recommends Notary Project tooling and supports Key Vault-backed signing; Docker Content Trust is being retired and is not selected: [Sign and verify OCI artifacts](#).

The provenance statement records Git commit, repository, source bundle digest, CI workflow identity, compiler image digest, dependency lock, build parameters, timestamps, SBOM digest, and output digest. Timestamps appear in provenance, never in canonical plan content.

13. Approval and separation of duties

Approval policy is derived from semantic change classification. A policy restriction, bug fix without authority change, or non-executable documentation update has a lower approval set than a new side effect, expanded data class, new vendor, new model task, weaker evidence requirement, or broader release cohort.

Role	Allowed actions	Prohibited combination
Package author	Propose source and respond to diagnostics	Cannot approve or sign own release
Domain owner	Approve KYC behavior and evaluation thresholds	Cannot mutate signing key permissions
Policy owner	Approve policy semantics and authority expansion	Cannot operate compiler identity
Security approver	Approve new data flow, network destination, or effect class	Cannot promote without domain approval
Evaluation service	Produce signed result candidate	Cannot waive failed threshold
Release manager	Validate gates, sign envelope, and promote	Cannot modify source or attestation
Emergency revoker	Revoke a digest and fence runtime	Cannot create a replacement release alone
Auditor	Read source-to-release evidence	No mutation permissions

Azure RBAC groups map to these roles. Workload identities use managed identity. Human production roles are eligible through Privileged Identity Management, require multifactor authentication and justification, and have short activation duration. Database roles use stored procedures or narrowly scoped permissions; the API never connects as server administrator.

14. Release, canary, promotion, and rollback

A release is a transaction across authoritative metadata and a small pointer projection. It never copies or modifies plan content.

1. Freeze the candidate dependency closure and verify all content digests.
2. Confirm the required evaluation attestation is passing, unexpired, and bound to the exact closure.
3. Confirm required human and machine approvals and their separation-of-duty rules.
4. Verify OCI signatures, SBOM, provenance, vulnerability policy, region, and assurance constraints.
5. Create and sign the canonical release envelope.
6. Insert the RELEASED record and audit event.
7. Create a canary scope with an explicit cohort, time window, and guardrails.
8. Publish the pointer using an ETag compare-and-swap transaction.
9. Observe runtime telemetry and business outcome guardrails.
10. Expand cohort by new pointer generation until production scope is reached.

App Configuration holds a projection of active pointer generations for low-latency resolution. Snapshot references can point to immutable snapshots and switch their target dynamically, as described in [Azure App Configuration snapshot references](#). This design permits a reference change only after the registry release transaction. Runtime auto-refresh cannot alter an in-flight case's authority; the case record continues to use its pinned digest.

Rollback creates a new pointer generation that references a previously signed, non-revoked release. It does not delete the bad release or restore a database backup. The action is idempotent, records the observed prior ETag, verifies compatibility with cases that have not started, and preserves a complete audit chain.

15. Revocation and emergency control

Revocation takes precedence over pinning. It is used for compromised keys, unsafe policies, vulnerable tools, invalid evidence producers, defective models, regulatory orders, or critical evaluation escapes. Deprecation prevents new adoption after a date; revocation immediately changes execution eligibility.

The revoker writes a signed revocation record in PostgreSQL and immutable Blob storage, emits a high-priority event on a dedicated Service Bus topic, updates the regional revocation projection, and triggers blast-radius analysis. Runtime resolvers poll and subscribe. Maximum revocation staleness is 60 seconds; a resolver unable to refresh beyond that bound fails closed for new high-assurance dispatches.

At runtime, revocation is checked when resolving a new case, before each command dispatch, and before committing any externally meaningful result. The response can stop, place the case in manual review, or migrate it to a signed replacement plan according to the revocation directive. Emergency migration never silently reinterprets prior evidence.

Key compromise has a dedicated runbook: disable affected key version, publish key and release revocations, halt promotion, rotate trust material, identify all signed subjects, re-sign only after evidence review, and verify every runtime has advanced its revocation generation.

16. APIs and events

APIM exposes versioned internal APIs. Every mutating command requires an idempotency key, authenticated Entra identity, correlation ID, and explicit expected version where a mutable lifecycle record is involved.

Endpoint	Semantics	Key response
POST /v1/packages:ingest	Store verified source bundle and manifest	Package digest and operation ID
POST /v1/builds	Compile a source and lock for an overlay set	Build ID
GET /v1/builds/{id}	Read state, diagnostics, outputs, and lineage	Build representation
POST /v1/evaluations	Evaluate an exact plan dependency closure	Evaluation run ID
POST /v1/approvals	Record scoped approval over a subject digest	Approval ID
POST /v1/releases	Assemble, verify, and sign candidate release	Release ID and envelope digest
POST /v1/releases/{id}:promote	Create new active-pointer generation	Pointer generation and ETag
POST /v1/scopes/{scope}:rollback	Point to approved prior release	New pointer generation
POST /v1/revocations	Revoke artifact, release, key, or capability scope	Revocation ID and generation
GET /v1/resolve	Resolve approved release for scope and context	Signed envelope and cache metadata
GET /v1/graph/impact	Return transitive dependents and active use	Impact report
GET /v1/explain/{digest}	Explain compilation, overlays, authority, and release gates	Structured explanation

Event subjects use `harness.control.<aggregate>.<event>`, including `build.succeeded`, `evaluation.passed`, `release.signed`, `pointer.activated`, `artifact.deprecated`, and `subject.revoked`. Payloads carry identifiers, digests, generations, scopes, correlation data, and schema version. They do not carry package bytes or secrets.

17. Concurrency, idempotency, and failure recovery

The database is the authority for command acceptance and state. A mutating API transaction inserts or reuses an idempotency record, changes aggregate state with an expected version, appends the audit event, and writes the outbox message. Duplicate requests return the original result. Queue delivery is at least once; consumers use inbox deduplication by event ID.

Compiler and evaluation jobs use leases with fencing tokens. A stale worker may upload content-addressed bytes but cannot publish a build or attestation after its lease is superseded. Artifact upload uses temporary paths, digest verification, and atomic registration. Orphan temporary objects are swept after a retention interval.

Promotion uses compare-and-swap on both database pointer generation and App Configuration ETag. If the projection update fails after the registry transaction, reconciliation completes it. A resolver validates the signed generation and never trusts a pointer that names a release absent from the registry cache.

Service Bus dead-letter queues are monitored. Replay is safe because events are facts and consumers are idempotent. Operators repair the cause, record an incident reference, and replay by event ID range; they do not edit consumer projections directly.

18. Azure topology and configuration

The initial production topology uses East US 2 as primary and Central US as recovery candidate, subject to service availability and organizational region policy. Non-production can use the same primary region with separate subscriptions, networks, identities, keys, registries, storage, databases, and Foundry projects. No production signing key exists in non-production.

Azure resource	Production baseline	Configuration
Management groups	Platform, NonProd, Prod, Security	Azure Policy inherited; deny public PaaS exposure and unapproved regions
Resource groups	network, control, data, security, observability per region	Resource locks on shared production data and security resources
Virtual network	Hub and spoke	Private DNS links; controlled egress through Azure Firewall; no worker public IPs
APIM	Internal mode or approved private ingress pattern	Entra JWT validation, mTLS for selected automation, quotas, schema validation, diagnostics
Container Apps environment	Workload profiles	Zone redundancy where supported; internal ingress; managed identities; min replicas for APIs and revoker
Service Bus Premium	Namespace per environment	Zone redundancy, private endpoint, duplicate detection, sessions only where ordering key is required
PostgreSQL Flexible Server	General Purpose or Memory Optimized after load test	Private access, zone-redundant HA, PITR, Entra auth, TLS, diagnostic settings
Storage account	GPv2 or Blob per artifact class	Private endpoint, public access disabled, versioning, soft delete, immutability for release evidence
ACR Premium	Registry per environment	Private endpoint, admin user disabled, content trust via Notation, retention and quarantine workflow
App Configuration	Store per environment and region	Private endpoint, local auth disabled, snapshots and references, only release projector writes production pointers
Key Vault Premium	Vault per environment and purpose	HSM-backed signing key, RBAC, private endpoint, purge protection, rotation policy
Foundry	Project per environment and region	Private networking where supported, immutable version capture, least-privilege deployer identity
Azure Monitor	Log Analytics plus Application Insights	Private Link Scope if required, diagnostic settings, alert rules, immutable audit export

Example production naming follows the organization convention rather than becoming an API contract:

Subscription: sub-harness-prod-us

Resource groups:

- rg-harness-control-eus2
- rg-harness-data-eus2
- rg-harness-security-eus2
- rg-harness-observability-us

Resources:

- cae-harness-control-eus2
- sb-harness-control-eus2
- psql-harness-registry-eus2
- stharnessartifactseus2
- acrharnessprodus
- appcs-harness-prod-eus2
- kv-harness-sign-prod-eus2

Key configuration values are maintained in IaC and policy, not in this narrative. Initial sizing is API min/max replicas 2/20, compiler concurrent jobs 10 per workload profile, release manager 2 replicas, revoker 2 replicas, PostgreSQL 4 to 8 vCores after load testing, Service Bus Premium 1 messaging unit with autoscale assessment, and ACR Premium. Load tests determine final values.

19. Network and identity flows

Inbound authoring automation reaches APIM over the approved private or enterprise ingress path and authenticates with federated workload identity. APIM forwards the caller claims; the Control API reauthorizes the action and records the identity. Workers obtain managed-identity tokens directly from Entra and use private endpoints for PostgreSQL, Blob, ACR, App Configuration, Key Vault, Service Bus, and supported Foundry endpoints.

Firewall rules allow only named destinations: Azure control APIs needed for deployment, approved package mirrors, vulnerability and signature services, and Foundry endpoints. Compiler jobs have no general outbound internet. Dependency artifacts are pre-fetched through an approved mirror and verified by digest. DNS resolution is private and centrally logged.

Each service gets a separate user-assigned managed identity. The compiler can read source and dependency artifacts and write candidate artifacts, but cannot sign, approve, promote, or read production secrets. The evaluator can invoke test deployments and write evaluation evidence. The signer can invoke one key's sign operation and read the release envelope; it cannot alter source or pointers. The projector can update the defined App Configuration prefix but cannot sign releases.

20. CI/CD and infrastructure pipeline

Infrastructure and control-plane software use independent pipelines. Infrastructure is Bicep or Terraform with reviewed modules, environment parameter files, static security scanning, what-if or plan review, policy checks, and drift detection. Application images are built once, scanned, signed, and promoted by digest across environments.

```
pull request
-> format + unit + schema + policy tests
-> safe compile + authority diff
-> IaC lint/security/what-if where applicable
-> build compiler/API/worker images
-> SBOM + vulnerability gate + OCI signature
-> deploy nonprod by digest
-> integration + fault + replay + performance tests
-> publish candidate release evidence
-> production approval
-> deploy production by same image digests
-> smoke test + observe + automatic rollback threshold
```

The domain release pipeline is separate:

```
commit -> source bundle -> schema validation -> deterministic compile
-> authority/data-flow/dependency reports -> policy and contract tests
-> golden/adversarial/metamorphic/fault evaluations
-> ACR and Foundry immutable versions -> EvaluationAttestation
-> approvals -> release-envelope signing -> canary pointer
-> shadow/guardrails -> staged production promotion
```

GitHub Actions or Azure DevOps uses OIDC workload federation; long-lived client secrets are prohibited. Production jobs target a protected environment and run from an allowlisted reusable workflow. The pipeline submits commands to the Control API rather than writing database rows, storage objects, or App Configuration keys directly.

21. Observability, audit, and service objectives

All services emit OpenTelemetry traces, metrics, and structured logs with operation ID, build ID, release ID, plan digest prefix, package, environment, region, stage, state transition, and result code. Logs never include secrets, raw evaluation data, prompts, documents, or unrestricted model output.

Objective	Proposed target	Alert condition
Registry resolve availability	99.95 percent monthly in primary region	Multi-window burn-rate alert
Cached resolution latency	p95 under 50 ms	15-minute p95 above 75 ms
Uncached resolution latency	p95 under 300 ms	15-minute p95 above 500 ms
Revocation propagation	99.9 percent under 60 seconds	Any production resolver exceeds 60 seconds
Promotion projection	99 percent under 30 seconds	Registry and App Configuration generations diverge for 60 seconds
Compile queue start	p95 under 2 minutes	Backlog age above 5 minutes
Determinism	100 percent identical digest on replay	Any mismatch pages platform engineering
Audit completeness	100 percent lifecycle transitions linked to identity and evidence	Reconciliation detects a gap

Dashboards cover build throughput and failures by diagnostic code, evaluation pass rate and flaky tests, authority expansions, approval age, signing errors, pointer generations, release cohort, revocation lag, registry cache age, outbox lag, DLQ depth, dependency blast radius, and Azure resource saturation.

Audit events use a monotonic aggregate sequence and hash chaining in immutable daily exports. A daily reconciliation verifies PostgreSQL records against Blob manifests, signatures, App Configuration generations, ACR digests, Foundry versions, and Key Vault key versions.

22. Resilience and disaster recovery

The design distinguishes runtime continuity from authoring continuity. Runtime resolvers hold verified releases and revocations in regional caches so a control-plane outage does not stop already authorized work inside cache bounds. New high-assurance work fails closed when the revocation view is stale beyond 60 seconds.

PostgreSQL uses zone-redundant HA and tested point-in-time restore. Blob uses the approved redundancy option plus immutable release evidence. ACR geo-replicates if the runtime recovery region requires local pull. App Configuration snapshots and pointer projections are reproducible from the registry. Service Bus messages are recoverable from authoritative outbox records.

Proposed control-plane targets are RPO at most 5 minutes for mutable lifecycle metadata and RTO at most 60 minutes for authoring and release operations. Revocation capability receives a stricter recovery path: pre-provisioned regional service, replicated trust material, and runbook target of 15 minutes. Signing may remain unavailable during a regional event; existing signed releases continue to verify.

Quarterly exercises restore PostgreSQL to an isolated subscription, reconstruct projections, verify a sample of artifacts and signatures, rotate a test signing key, revoke a plan, promote a prior release, and prove runtime cache behavior. Recovery is complete only when registry, pointer, revocation, ACR, Blob, Foundry, and audit generations reconcile.

23. KYC-specific control-plane rules

The United States KYC package defines customer types, policy outcomes, evidence requirements, and vendor capabilities without embedding vendor credentials or production endpoints. Initial customer types are individual and sole proprietor; legal-entity KYB remains a separate package or explicitly versioned extension.

Required KYC compile rules include:

- An approval or rejection outcome must cite policy rules and evidence schemas; unreasoned model output cannot be a decision basis.
- Sanctions, PEP, adverse media, identity, SSN, document, liveness, fraud, and address capabilities declare jurisdiction, data class, freshness, assurance, and fallback behavior.
- Restricted identifiers cannot be sent to a model profile unless that exact purpose and region are allowed.
- Each vendor command declares whether it is read-only, externally visible, billable, or irreversible.
- A manual-review path is required for inconclusive evidence, vendor unavailability beyond retry budget, policy conflict, and assurance shortfall.
- Policy bundles distinguish deterministic eligibility rules from model-assisted extraction or summarization.
- Evidence reuse across plan versions requires matching schema, freshness, provenance, and migration policy.
- No released path can produce approved when a mandatory evidence producer is absent or revoked.

Golden suites cover known document types, supported and unsupported states, name and address variations, SSN mismatch, duplicate identity, sanctions and PEP matches, deceased identity, synthetic identity indicators, expired documents, and vendor partial responses. Adversarial suites attempt evidence forgery, prompt injection inside OCR text, tool-name spoofing, schema smuggling, and manipulation of policy citations.

24. Implementation plan

Phase	Duration	Deliverables and exit criteria
0. Decisions and threat model	2 weeks	ADRs for authority, canonicalization, signing, registries, regions, tenancy, data classification, and failure posture; approved threat model
1. Azure foundation	3 weeks	Subscriptions, networks, private DNS, APIM, Container Apps, PostgreSQL, Storage, ACR, Service Bus, App Configuration, Key Vault, monitoring, policies, and IaC validation
2. Registry core	4 weeks	Artifact, package, build, dependency, audit, idempotency, outbox, and query APIs; backup and reconciliation tested
3. Compiler MVP	6 weeks	Safe parser, schemas, dependency lock, overlays, capability and policy compiler, permission and graph analysis, canonicalization, diagnostics, deterministic replay
4. Evaluation system	5 weeks	Suite manifest, runner adapters, evidence store, threshold engine, attestation format, KYC golden/adversarial/fault suites
5. Signing and release	4 weeks	Provenance, SBOM binding, Key Vault signer, approvals, release state machine, App Configuration projection, canary and rollback
6. Foundry and ACR integration	3 weeks	OCI signing verification, agent version deployment, immutable identifiers, deployment attestation, runtime resolver contract
7. Revocation and impact	3 weeks	Dependency projection, impact API, signed revocation, 60-second runtime propagation, compromised-key runbook
8. KYC pilot	4 weeks	Individual KYC package, evaluation thresholds, canary cohort, shadow comparison, production readiness review
9. Hardening and DR	3 weeks	Performance, fault injection, restore, regional recovery, audit evidence, SLO alerts, operational handoff

The phases overlap after interfaces stabilize. A practical minimum team is one control-plane lead, two compiler and registry engineers, two platform engineers, two evaluation/domain engineers, one security engineer, one SRE, and fractional KYC policy and compliance owners.

25. Prioritized engineering backlog

ID	Work item	Acceptance criterion
HC-01	Define canonical HarnessPlan JSON profile	Independent implementations produce identical bytes and digest for conformance fixtures
HC-02	Implement safe YAML parser limits	Malicious tags, aliases, duplicate keys, oversized and deeply nested inputs fail with stable codes
HC-03	Build immutable dependency resolver and lock	All transitive dependencies resolve by digest; mutable references fail
HC-04	Implement overlay engine and origin map	Every resolved field has source lineage; unauthorized expansion fails
HC-05	Build capability qualifier	Region, data, assurance, interface, certification, and availability constraints produce deterministic choice or error
HC-06	Compile policy and obligations	Golden allow, deny, and obligation cases are complete and explainable
HC-07	Derive least-privilege node permissions	Runtime can validate every command against compiled authority
HC-08	Implement data-flow and workflow analysis	Residency violations, missing producers, cycles, unsafe effects, and unreachable outcomes fail
HC-09	Create registry API and artifact verification	Metadata cannot register until content digest and provenance verify
HC-10	Add evaluation manifest and attestation	Attestation binds complete closure, metrics, thresholds, environment, and expiry
HC-11	Integrate ACR Notation verification	Unsigned or incorrectly signed OCI artifact cannot build or release
HC-12	Integrate Key Vault release signing	Only fully gated release envelope can reach sign operation
HC-13	Implement pointer generations and reconciliation	Promotion and rollback are atomic, idempotent, and recover after projection failure
HC-14	Implement signed revocation	All production resolvers enforce revocation in under 60 seconds
HC-15	Build dependency impact query	Active releases and pinned cases are returned for any transitive dependency
HC-16	Deliver KYC pilot package	Individual KYC passes policy, golden, adversarial, fault, shadow, and canary gates

26. Verification strategy

Unit tests cover canonicalization, schema validation, overlay operators, dependency resolution, policy compilation, permission derivation, graph algorithms, signature envelopes, state transitions, and idempotency. Property tests generate equivalent source orderings and assert identical canonical bytes. Fuzz tests target YAML parsing, schemas, graph size, and diagnostic stability.

Integration tests use real PostgreSQL, Service Bus, Blob, ACR, App Configuration, and Key Vault instances in an isolated Azure test subscription. They verify private DNS, managed identities, digest checking, signer restrictions, outbox replay, pointer reconciliation, and runtime resolution. Foundry adapters test immutable version capture and invoke behavior.

End-to-end tests exercise commit through production-like canary: compile a KYC package, run evaluations, approve, sign, promote, start a case with a pinned digest, promote a successor, prove the first case remains pinned, revoke a shared tool, and verify both new dispatch and result commit are fenced.

Fault injection terminates compiler workers, duplicates and reorders messages, expires leases, blocks Blob or App Configuration, fails signing, makes Foundry deployment time out, introduces a database failover, and delays revocation events. Success means no invalid release is signed, no pointer advances without an authoritative release, no stale worker

publishes, and every failure is observable and recoverable.

Performance tests measure dependency graphs up to the forecast three-year size, burst pull-request builds, concurrent evaluation suites, resolve load, and mass-revocation impact. Production readiness requires capacity at twice expected peak with target latency and no exhausted connection, queue, or API limits.

27. Operational runbooks

The initial runbook set covers compile backlog, deterministic digest mismatch, failed evaluation infrastructure, flaky suite quarantine, signature failure, expired attestation, promotion conflict, registry/App Configuration divergence, ACR signature failure, Foundry version failure, database failover, outbox backlog, DLQ replay, stale resolver cache, artifact corruption, plan revocation, dependency mass revocation, signing-key compromise, and regional recovery.

Every runbook states detection, customer and release impact, immediate containment, diagnostic queries, permitted repair operations, rollback or revocation criteria, evidence to preserve, escalation contacts, and completion checks. Direct database or pointer edits are excluded. Emergency actions go through the revocation or release APIs and preserve the audit chain.

28. Architecture decisions and definition of done

The first architecture decision records approve PostgreSQL as lifecycle authority, Blob and ACR as content-addressed artifact stores, App Configuration as a pointer projection, Key Vault as signer, canonical JSON plus SHA-256 as plan identity, OCI Notation signatures, at-least-once messaging with idempotent consumers, digest-pinned cases, and revocation precedence.

The Harness Control Plane is ready for the KYC production pilot when all of the following are demonstrated:

- Two clean environments can be created from IaC with private access, managed identity, policy, logging, backup, and no manual resource configuration.
- Independent compiler replays produce byte-identical plans and stable diagnostic codes.
- A release cannot be signed without exact passing attestation, required approvals, verified provenance, acceptable SBOM and vulnerability status, and non-revoked dependencies.
- Runtime resolution validates signature, digest, scope, pointer generation, time constraints, and revocation.
- Promotion, rollback, reconciliation, and duplicate delivery pass fault tests.
- Revocation fences production dispatch and commit within 60 seconds and reports complete blast radius.
- KYC golden, adversarial, metamorphic, fault, shadow, and canary gates meet domain-approved thresholds.
- Restore and regional exercises meet RPO and RTO, and all authoritative and projected generations reconcile.
- Security, KYC policy, privacy, platform, SRE, and audit owners approve the evidence package and operational ownership.

The result is a control plane in which editable configuration becomes executable only after deterministic compilation, explicit authority analysis, evidence-backed evaluation, independent approval, cryptographic signing, controlled promotion, and continuously enforceable revocation.